

Week 2: Context Free Grammars

* 1. Consider the following CFG G with start symbol R :

$$\begin{aligned} R &::= XRX \mid S \\ S &::= aTb \mid bTa \\ T &::= XTX \mid X \mid \epsilon \\ X &::= a \mid b \end{aligned}$$

- (a) What are the non-terminals of G ?
- (b) What are the terminals of G ?
- (c) Give three strings in $L(G)$.
- (d) Give three strings not in $L(G)$.
- (e) True or false: $T \rightarrow aba$.
- (f) True or false: $T \rightarrow^* aba$.
- (g) True or false: $T \rightarrow T$.
- (h) True or false: $T \rightarrow^* T$.
- (i) True or false: $XXX \rightarrow^* aba$.
- (j) True or false: $X \rightarrow^* aba$.
- (k) True or false: $T \rightarrow^* XX$.
- (l) True or false: $T \rightarrow^* XXX$.
- (m) True or false: $S \rightarrow^* \epsilon$.

Solution

- (a) R, S, T, X
- (b) a, b
- (c) ab, ba, aab

The syntax of the *Brischeme* programming language (ignoring whitespace) is given by the CFG with:

- Terminals: $0, 1, \dots, 9, a, b, \dots, z, A, B, \dots, Z, _ , !, ? , < , = , + , - , * , / , (,)$, define, lambda, not, or, and, #t, #f.
- Nonterminals: *Prog*, *Form*, *SExpr*, *Ident*, *Literal*, *Num*, *Bool*, *Digit*, *LChar*, *IdChar*, *Primop*.
- The production rules are:

$$\begin{aligned}
 \textit{Prog} &::= \textit{Form}^* \\
 \textit{Form} &::= \textit{SExpr} \\
 &\quad | \quad (\text{define } \textit{Ident} \textit{SExpr}) \\
 \textit{SExpr} &::= \textit{Literal} \\
 &\quad | \quad \textit{Ident} \\
 &\quad | \quad (\textit{SExpr} \textit{SExpr}^*) \\
 &\quad | \quad (\textit{Primop} \textit{SExpr}^*) \\
 &\quad | \quad (\text{lambda } (\textit{Ident}^*) \textit{SExpr}) \\
 \\
 \textit{Ident} &::= \textit{LChar} \textit{IdChar}^* \\
 \textit{LChar} &::= a | b | \dots | z \\
 \textit{IdChar} &::= a | b | \dots | z | A | B | \dots | Z | ! | ? | _ \\
 \\
 \textit{Literal} &::= \textit{Bool} | \textit{Num} \\
 \textit{Bool} &::= \text{\#t} | \text{\#f} \\
 \textit{Num} &::= \textit{Digit} \textit{Digit}^* \\
 \textit{Digit} &::= 0 | 1 | \dots | 9 \\
 \\
 \textit{Primop} &::= + | - | * | / | \text{and} | \text{or} | \text{not} | < | =
 \end{aligned}$$

Figure 1: *Brischeme* (ignoring whitespace).

- (d) aa, bb, ϵ
- (e) false
- (f) true
- (g) false
- (h) true
- (i) true
- (j) false
- (k) true
- (l) true
- (m) false

* 2. Figure 1 contains a grammar for the Brscheme language from this week's lab sheet.

You will need to read the section **Grammars can Express Sequences** from the end of in the course notes to understand the notation $Form^*$, $SExpr^*$, $Ident^*$ and $IdChar^*$ from this grammar. Unfortunately, I did not have time to cover this in the lecture.

Which of the following are valid Brscheme programs (i.e. strings in the language of that grammar)? You do not have to give the derivations (but you should work through them in your head).

- (a) (define x 32) (+ 4 x)
- (b) (define x 32) (+ 4 y)
- (c) (define x 32) (+ 4 5x)
- (d) (define aC3! 32) (+ 4 aC3!)
- (e) (define AC3! 32) (+ 4 AC3!)
- (f) (+ (define x 32) x 4)
- (g) (lambda x (+ x x))
- (h) (define f (lambda (xy) (+ x (* 2 y))))
- (i) (lambda (x b) (+ x (not b)))
- (j) (lambda (x b) (if x))
- (k) (lambda (x b) ((if b + -) 2 3))
- (l) ((lambda ()) 3))

Solution

This question is a little unfair because I have been inconsistent on how I use whitespace (a question with such ambiguity would not appear in an exam). On the one hand, I told you that we essentially ignore whitespace for now, and on the other it seems to be crucial to understand some of these examples. We will discuss whitespace in Week 3.

- (a) yes
- (b) yes
- (c) no – identifiers cannot start with a digit. However, if we really ignore whitespace then it is possible to parse `(+ 4 5x)` as `(+ 4 5 x)` which is a valid s-expression.
- (d) yes
- (e) no – identifiers must start with a lowercase letter.
- (f) no – `define` is not derivable from *SExpr*, so cannot be nested in another s-expression. However, if we really ignore whitespace, then this could be understood as `(+ (define x 32) x 4)`, which is a valid s-expression.
- (g) no – formal parameters to a lambda function must be parenthesized. However, if we really ignore whitespace, then this could be understood as `(lambda x (+ x x))`, which is a valid s-expression.
- (h) yes
- (i) yes
- (j) yes
- (k) no – `+` and `-` are not derivable from *SExpr*
- (l) yes

** 3. Design CFGs for the following programming language lexemes over the ASCII alphabet. You will find it convenient to use abbreviations like $\dots$ to help present the expressions compactly.

- (a) A *C program identifier* is any string of length at least 1 containing only letters ('a'–'z', lower and uppercase), digits ('0'–'9') and the underscore, and which begins with a letter or the underscore.
- (b) An *integer literal* is any string taking one of the following forms:
 - a non-empty sequence of digits (decimal)
 - a non-empty sequence of characters from '0'–'9', 'a'–'e' (upper or lowercase) that are preceded by "0x" (hexadecimal)
 - a non-empty sequence of bits '0' and '1' that are preceded by "0b" (binary)

Solution

- (a) This is one way to describe it, I will write terminal symbols in terminal type to distinguish them:

$$\begin{aligned} S &::= LM \\ M &::= LM \mid DM \mid \epsilon \\ L &::= a \mid b \mid \dots \mid z \mid _ \mid A \mid B \mid \dots \mid Z \\ D &::= 0 \mid 1 \mid \dots \mid 9 \end{aligned}$$

- (b) One straightforward way is as follows (here I use terminal type for the terminal symbols to distinguish them from nonterminals):

$$\begin{aligned} S &::= D \mid H \mid B \\ D &::= ND \mid N \\ N &::= 0 \mid 1 \mid \dots \mid 9 \\ H &::= 0xG \\ G &::= AG \mid A \\ A &::= N \mid a \mid b \mid \dots \mid e \mid A \mid B \mid \dots \mid E \\ B &::= 0bZ \\ Z &::= YZ \mid Y \\ Y &::= 0 \mid 1 \end{aligned}$$

- * 4. Consider the following grammar, which describes the structure of statements in an imperative programming language.

$$\begin{aligned} Prog &::= Stmt Stmts & (1) \\ Stmt &::= \text{if exp then } Stmt \text{ else } Stmt & (2) \\ & \mid \text{while exp do } Stmt & (3) \\ & \mid \text{skip} & (4) \\ & \mid \text{id} \leftarrow \text{exp} & (5) \\ & \mid \{ Stmt Stmts \} & (6) \\ Stmts &::= ; Stmt Stmts & (7) \\ & \mid \epsilon & (8) \end{aligned}$$

In it expressions appear only as a terminal symbols *exp* because the structure of expressions is not important to the exercises. In total, the terminal symbols are: *if*, *then*, *else*, *while*, *do*, *skip*, *id*, *exp*, the left and right braces, the end-of-input marker and the semicolon. The rules are numbered to make constructing the parse tables easier. The language of this grammar (start symbol *Prog*) includes strings such as:

while *exp* do *id* $\leftarrow$ *exp*

i.e. strings that show the control structure of the program without specifying the particular expressions involved.

The nullable, first and follow maps for the nonterminals in this grammar are as follows:

Nonterminal	Nullable?	First	Follow
Prog	×	if, while, skip, id, {	
Stmt	×	if, while, skip, id, {	else, ;, }
Stmts	✓	;	}

- (a) For each rule $X ::= \alpha$ numbered (1) – (8), compute $\text{First}(\alpha)$, the set of terminal symbols that can start any string derivable from the rule right-hand side α .

(b) Construct the parsing table for the grammar.

(c) Is the grammar LL(1)?

Solution

(a)

$\text{First}(\text{Stmt Stmt}) = \text{if, while, skip, id}$
 $\text{First}(\text{if exp then Stmt else Stmt}) = \text{if}$
 $\text{First}(\text{while exp do Stmt}) = \text{while}$
 $\text{First}(\text{skip}) = \text{skip}$
 $\text{First}(\text{id} \leftarrow \text{exp}) = \text{id}$
 $\text{First}(\{\text{Stmt Stmt}\}) = \{$
 $\text{First}(\text{; Stmt Stmt}) = \text{;}$
 $\text{First}(\epsilon) =$

(b)

Nonterminal	if	exp	then	else	while	do	skip	id	;	{	}
Prog	1				1		1	1		1	
Stmt	2				3		4	5		6	
Stmts									7		8

(c) Yes.

* 5. Consider now the following grammar:

$\text{Prog} ::= \text{Stmt Stmt} \quad (1)$
 $\text{Stmt} ::= \text{if bexp then Stmt else Stmt} \quad (2)$
 $\quad \quad \quad | \text{while bexp do Stmt} \quad (3)$
 $\quad \quad \quad | \text{skip} \quad (4)$
 $\quad \quad \quad | \text{id} \leftarrow \text{aexp} \quad (5)$
 $\quad \quad \quad | \text{Stmt Stmt} \quad (6)$
 $\text{Stmts} ::= \text{; Stmt Stmt} \quad (7)$
 $\quad \quad \quad | \epsilon \quad (8)$

This grammar is the same as the previous one, except that braces have been removed in rule 6. The Nullable, First and Follow maps for this grammar can be tabulated as follows.

Nonterminal	Nullable?	First	Follow
Prog	×	if, while, skip, id	
Stmt	×	if, while, skip, id	;, else
Stmts	✓	;	;, else

(a) For each rule $X ::= \alpha$ numbered (1) – (8), compute $\text{First}(\alpha)$, the set of terminal symbols that can start any string derivable from the rule right-hand side α .

(b) Construct the parsing table for this grammar.

(c) Is the grammar LL(1)?

Solution

(a)

- (1) $\text{First}(\text{Stmt Stmt}) = \text{if, while, skip, id}$
- (2) $\text{First}(\text{if exp then Stmt else Stmt}) = \text{if}$
- (3) $\text{First}(\text{while exp do Stmt}) = \text{while}$
- (4) $\text{First}(\text{skip}) = \text{skip}$
- (5) $\text{First}(\text{id} \leftarrow \text{exp}) = \text{id}$
- (6) $\text{First}(\text{Stmt Stmt}) = \text{if, while, skip, id}$
- (7) $\text{First}(\text{; Stmt Stmt}) = \text{;}$
- (8) $\text{First}(\epsilon) =$

(b) The parsing table is as follows:

Nonterminal	\$	if	bexp	then	else	while	do	skip	id	aexp	;
Prog		1				1		1	1		
Stmt		2, 6				3, 6		4, 6	5, 6		
Stmts	8				8						7, 8

(c) No.

** 6. Give CFGs for the following languages. The later parts are more difficult than 2-star.

(a) All odd length strings over $\{a, b\}$.

(b) All strings over $\{a, b\}$ that contain aab as a substring.

(c) The set of strings over $\{a, b\}$ with more a than b . Hint: every string w with at least as many a as b (possibly the same number of a as b) can be characterised inductively as follows. Either:

- w is just a
- or, w is of shape avb with v containing at least as many a as b
- or, w is of shape bva with v containing at least as many a as b
- or, w is of shape v_1v_2 with v_1 and v_2 each separately containing at least as many a as b
- or, w is the empty string

(d) The complement of the language $\{a^n b^n \mid n \geq 0\}$ over $\{a, b\}$. Hint: express "not of shape $a^n b^n$ for some n " into one or more positive (i.e. without using *not* or similar) conditions.

(e) $\{v\#w \mid v, w \in \{a, b\}^* \text{ and the reverse of } v \text{ is a substring of } w\}$, over $\{a, b, \#\}$.

(a)

$$\begin{aligned} S &::= XSX \mid X \\ X &::= a \mid b \end{aligned}$$

(b)

$$\begin{aligned} S &::= XSX \mid aab \\ X &::= a \mid b \mid \epsilon \end{aligned}$$

- (c) Here we dedicate a nonterminal T to describe the inductive characterisation given in the hint and then use S to force an extra a .

$$\begin{aligned} S &::= TaT \\ T &::= a \mid aTb \mid bTa \mid TT \mid \epsilon \end{aligned}$$

- (d) When you are asked to give the complement, or otherwise to describe something according to a negated condition, it is best to translate it immediately into one or more positive conditions because CFG have no direct way to express negation. In this case, not of shape $a^n b^n$ means either:

- The word has all the a before any b but there are strictly more a than b (nonterminal R),
- or, the word has all the a before any b , but there are more b than a (nonterminal T),
- or, the word has a b occurring before an a (non-terminal U).

$$\begin{aligned} S &::= R \mid T \mid U \\ R &::= AaRb \mid a \\ A &::= aA \mid \epsilon \\ T &::= aTbB \mid b \\ B &::= bB \mid \epsilon \\ U &::= XbXaX \\ X &::= aX \mid bX \mid \epsilon \end{aligned}$$

- (e) Strings in this language have the following shape: $v\#w_1v^Rw_2$ where w_1 and w_2 are completely arbitrary. We need to match v to v^R letter by letter, so this is best done by a single nonterminal (T). The “middle” of the string generated by this non-terminal (between the matching pieces of v and its reverse) will contain the $\#$ and, to the right of it, any arbitrary substring w_1 . Finally, the whole string has an arbitrary suffix w_2 . We can describe this as follows:

$$\begin{aligned} S &::= TX \\ X &::= aX \mid bX \mid \epsilon \\ T &::= aTa \mid bTb \mid \#X \end{aligned}$$